Informe del Proyecto “Amigos para Siempre, con Ude@Book”

Tanya Delgado

Facultad de Ingeniería, Universidad de Antioquia

Informática II

Aníbal Guerra

18 de octubre de 2026

Introducción

La propuesta de desafío contempla el desarrollo de una red social en C++, con interacción a través de la consola, en donde al ingresar, el usuario registrado puede administrar sus mensajes, solicitudes de amistad, ver perfiles de amigos con sus respetivas fotos y visualizar las métricas asociadas a la ejecución del programa.

Además, todo el desarrollo es basado en la POO con gestión de memoria dinámica, contando con restricciones en cuanto al uso de la STL u otras librerías que faciliten las funcionalidades a implementar.

Análisis para el Desarrollo

Teniendo en cuenta los requerimientos iniciales, podemos determinar que la solución puede dividirse en componentes según las funcionalidades requeridas, estos serían: usuario (datos asociados), amistades, fotos con sus respectivos álbumes, sugerencias de amistad, gestión de mensajería y gestor de archivos con medición de eficiencia.

Tenemos en cuenta que no se utilizan STL, se manipula la memoria dinámica, la forma de visualizar las imágenes es a través de su ruta pues no se almacenan directamente y la gestión de datos no se visualiza en consola, solo las mediciones al finalizar una funcionalidad.

Para la arquitectura de la solución bajo la POO, ‘RedSocial’ actúa como la clase que coordina las funcionalidades disponibles; ‘Album’, ‘Foto’, ‘Usuario’, ‘Mensaje’ y ‘Solicitud’ serían la lógica central; ‘GestorArchivos’ se encarga de relacionado a los archivos que emplea el programa; ‘MatrizAdyacencia’ sería la red de amigos; y las clases con listas dinámicas manejan el almacenamiento.

La matriz de adyacencia con memoria dinámica hace una representación de las amistades mediante posiciones binarias, siendo idónea para el calculo simplificado de los amigos en común. Además, la lista dinámica utiliza un arreglo dinámico que modifica su capacidad cuando llega al límite y almacenan punteros a objetos para evitar copias completas.

Diagrama de Clases

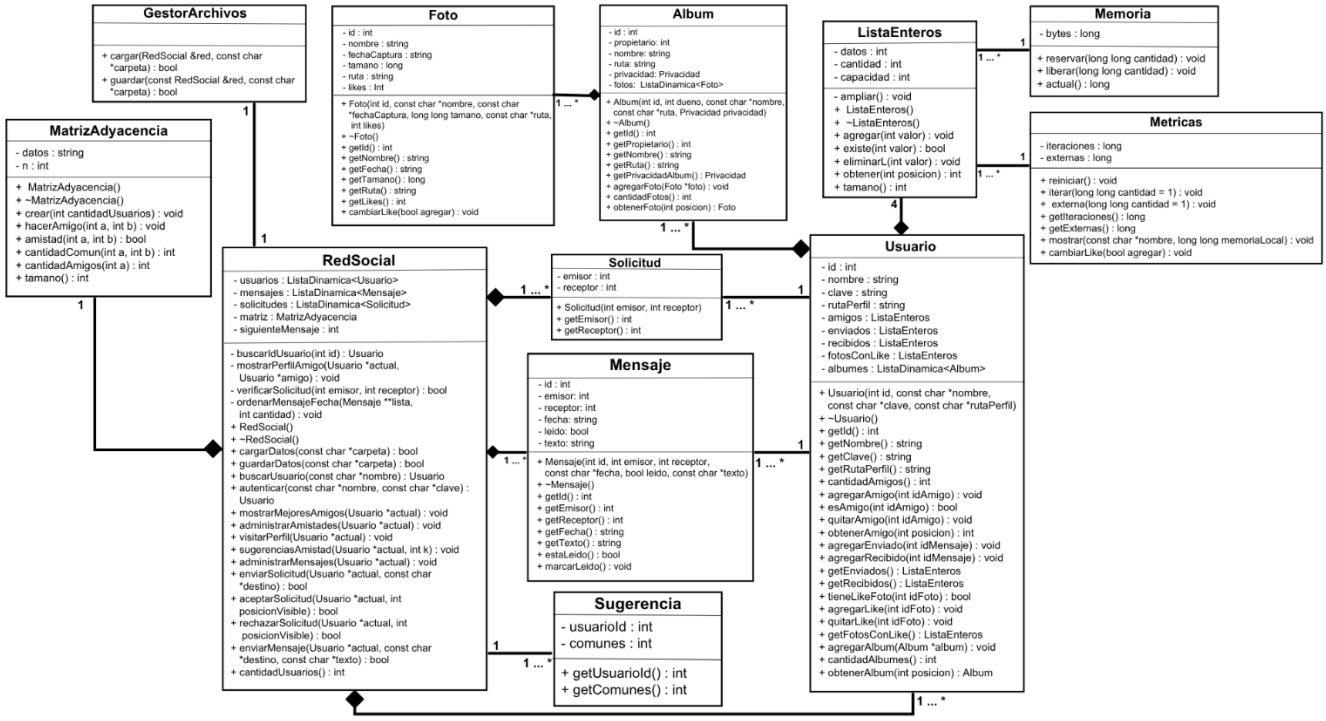


Figura 1. Diagrama de Clases Completo

Algoritmos Esenciales en el Desarrollo

Dado a que dividimos las funcionalidades en subprogramas para su adecuada ejecución, tenemos los siguientes:

Carga de Información. Para este se crean los usuarios, es necesario identificarlos para continuar; luego se enlazan las amistades, actualizando a la par la lista de amigos y la matriz; después irían los álbumes y fotos asociados entre sí; finalmente se reconstruyen solicitudes, mensajes y 'likes'. Todo el proceso de carga respeta las dependencias entre entidades.

Selección de Perfiles en el Inicio. A través de la matriz de adyacencia se calcula la cantidad de amigos en común entre el usuario autenticado y su lista de amigos, asignando la relevancia relativa así: 10 cuando tengan entre 0-10 comunes, 25 cuando sean entre 11-20 y 35 cuando sean más de 20; lo anterior representando las relaciones de 1, 2.5 y 3.5 establecidas. Entonces, se construye el conjunto teniendo en cuenta su relevancia, se genera valor aleatorio dentro del rango, se recorre hasta encontrar el usuario, excluyéndolo en la siguiente ronda, y se repite el proceso hasta completar los 3; en la actualización, se excluyen los 3 anteriores siempre que la cantidad de amigos lo permita.

Visualización de Perfiles. Para esto se valida que el perfil solicitado corresponda a un amigo del usuario, luego, se muestran sus álbumes disponibles, al seleccionarlo se muestra una fotografía y se permite mover hacia atrás o adelante en este, al llegar a un extremo se devuelve al otro para permitir una vista como 'carrusel'. Por otra parte, los 'likes' de cada foto se muestran a modo de contador, permitiendo verificar si ya el usuario se lo otorgó.

Gestión Solicitudes de Amistad. Cuando se envía una solicitud se busca el destinatario, validando que exista, no sea el mismo que el emisor, no sean amigos actualmente o que no haya una solicitud ya vigente hacia este; siendo lo anterior verdadero, se crea la nueva solicitud. Además, cuando se acepta una solitud, obtenemos quién la envió, se añade la relación entre ambos, actualizando matriz de adyacencia y la eliminación de la solicitud; al rechazar una solicitud, solo se elimina esta.

Generar Sugerencias de Amistad. Con la matriz de adyacencia podemos identificar los amigos de amigos; para cada usuario valido, que no sea él mismo o sus amigos, se calcula cantidad de relaciones, considerando solo los que tienen al menos un amigo en común; se

conserva solo los 10 primeros, ordenándolos de mayor a menor en cuantos a la cantidad de coincidencias, y se ofrece la opción de enviar solicitud.

Gestión de Mensajes. Para leer los mensajes, se buscan aquellos que tengan como receptor el usuario autenticado y no se hayan leído, mostrándolos todos en orden de fecha de envío, posteriormente se marcan como 'leídos'. También se tiene la búsqueda por rango de fecha, que permite seleccionar los mensajes con una fecha inicial y final, con un formato estándar de esta.

Mediciones. Las métricas registran las interacciones directas e indirectas de las operaciones realizadas, incluyendo llamados a componentes externos como valores aleatorios. Con Memoria se registran las reservas y liberaciones de memoria generadas por las estructuras establecidas; es así como al finalizar cada funcionalidad, se muestra una actualización de estas.

Problemas de Desarrollo Contemplados

Entre los problemas de desarrollo que implica el desafío, tenemos la limitación del uso de STL, por lo que se emplearon listas dinámicas y se maneja manualmente sus modificaciones; al emplear punteros se debe establecer quién crea y libera cada objeto, para evitar fugas de memoria; ante la posibilidad de generar repeticiones en la selección de los 3 amigos iniciales, se trabajará con una selección ponderada; también, las listas de amigos deben ser consistentes entre usuarios y la matriz.

En cuanto a los ‘likes’, se debe conservar la relación que determina que el usuario le haya dado ‘like’ a cada foto; además, se requiere la medición de memoria, dado a que no es posible establecer la memoria física, se visualiza la asociada a las reservas propias.

Consideraciones para la Implementación

Para el almacenamiento de los datos requeridos de los usuarios para el funcionamiento del programa, se emplearán archivos independientes dentro de una carpeta ‘datos_completos’ los cuales tienen separador de campos ‘|’; entre estos archivos se encuentran: usuarios.txt, solicitudes.txt, mensajes.txt, entre otros. Dado a que las fotos se almacenan mediante rutas, se reduce el tamaño de los datos a emplear; las rutas se almacenan en otro archivo de texto.

La matriz adyacente facilita el proceso de encontrar los amigos en común entre usuarios; también, se emplean referencias constantes y punteros cuando no se necesita copiar objetos completos, esto como parte de la eficiencia del programa; para mantener la coherencia en el almacenamiento permanente, primero se hace la carga de la información al iniciar la ejecución del programa y el guardado de datos después de las modificaciones.